EXPERIMENT -3

Student Name: Aditya Raj Bhakta **UID: 24BAI70782**
Branch: AIT-CSE (AIML) **Section/Group: 24AIT_KRG-1A**
Semester: 5
Subject Name: Database ManagementSystem **Subject Code: 24CSH-298**

Q1)
Youare given with the Employee schema; the task is to find the highest score for each respective department.

For eg: department D1 have the highest score as 5.28. Hence in the output, final updated score for D1 should be 5.28.
Same with the other departments i.e., D2, D3 and D4.

SOFTWARE REQUIREMENTS:

Database Management System:
Oracle Database Express Edition (Oracle XE) PostgreSQL Database

Database Administration Tool / Client Tool:
Oracle SQL Developer (for Oracle XE)
pgAdmin (for PostgreSQL)

CODE SCREENSHOTS:

```
update employee as e
set scores =(select max(scores) from employee
where Dept=e.dept
)
select EID, DEPT, SCORES from employee
```

EXPERIMENT -3

OUTPUT SCREENSHOTS:

	eid [PK] integer	dept character varying (10)	scores numeric (5,2)
1	1	D1	5.28
2	2	D1	5.28
3	3	D1	5.28
4	4	D2	8.00
5	5	D1	5.28
6	6	D2	8.00
7	7	D3	9.00
8	8	D4	10.20

LEARNING OUTCOMES:

After completing this experiment, I was able to:

- Learned how to group records using the GROUP BY clause.
- Understood the use of aggregate functions such as MAX() to find the highest value.
- Learned to retrieve department-wise summary information from a dataset.
- Gained experience in generating reports based on grouped data.

CONCLUSION:

This experimentdemonstrated how to use SQL aggregation to determine the highest employee scoreineach department. By grouping records by department and applying the MAX() function,the highest score for each department (D1, D2, D3, and D4) was successfully identified and displayed.

EXPERIMENT -3

Q2)
Acompany maintains records of products sold by different salespersons. The management wantsto prepare a sales performance report by identifying the salesperson(s) who generated thehighest overall sales revenue. If multiple salespersons have the same highest total sales amount, all of them must be included in the final report. Return the seller ID(s) satisfying this condition.

SOFTWARE REQUIREMENTS:

Database Management System:
Oracle Database Express Edition (Oracle XE) PostgreSQL Database

Database Administration Tool / Client Tool:
Oracle SQL Developer (for Oracle XE)
pgAdmin (for PostgreSQL)

CODE SCREENSHOTS:

```
select seller_id from orders
group by seller_id
having sum(amount)=(
select max(a.amts) from
(select seller_id, sum(amount) as amts from orders
group by seller_id) as a)
```

EXPERIMENT -3

OUTPUT SCREENSHOTS:

	seller_id integer
1	3

LEARNING OUTCOMES:

After completing this experiment, I was able to:

- Learned how to calculate total sales using the SUM() aggregate function.
- Understood how to group sales records by seller ID.
- Learned to identify all sellers with the maximum total sales, including ties.
- Gained experience in creating performance-based summary reports.

CONCLUSION:

This experiment demonstrated how to calculate the total sales revenue generated by each salesperson and identify the salesperson(s) with the highest overall sales. By using SQL aggregation and comparison techniques, all sellers with the maximum total sales amount were successfully included in the final report.

EXPERIMENT -3

Q3)
A company maintains employee salary records for different departments. The management wants to prepare a report containing employees whose salaries fall among the **top three highest distinct salary values** within their respective departments. If multiple employees receive the same qualifying salary, all such employees must be included in the final report. Display the department name, employee name, and salary for every qualifying employee.

SOFTWARE REQUIREMENTS:

Database Management System:
Oracle Database Express Edition (Oracle XE) PostgreSQL Database

Database Administration Tool / Client Tool:
Oracle SQL Developer (for Oracle XE)
pgAdmin (for PostgreSQL)

CODE SCREENSHOTS:

```
select d.name as department, e.name
as employee, e.salary as salary from employee as e
join department as d on
e.departmentid=d.id
where (
    SELECT COUNT(DISTINCT E2.SALARY)
    FROM EMPLOYEE AS E2
    WHERE E2.departmentId = e.departmentId
    AND
    E2.SALARY > e.SALARY
)<3
order by d.name
```

EXPERIMENT -3

OUTPUT SCREENSHOTS:

	department character varying (50)	employee character varying (50)	salary integer
1	IT	Joe	85000
2	IT	Max	90000
3	IT	Randy	85000
4	IT	Will	70000
5	Sales	Henry	80000
6	Sales	Sam	60000

LEARNING OUTCOMES:

After completing this experiment, I was able to:

- Learned how to rank records within each department using SQL ranking functions.
- Understood the concept of partitioning data by department.
- Learned to retrieve employees with the top three distinct salary values. Gained knowledge of handling salary ties while generating reports

CONCLUSION:

This experiment demonstrated how to identify employees whose salaries are among the top three distinct salary values within their respective departments. By using SQL ranking techniques and partitioning by department, all eligible employees, including those with tied salaries, were successfully included in the final report.